

Что изменилось в движке (ЛР1: Job System, база — ветка 3lab)

Коротко: взяла проект из ветки 3lab (ECS, Assimp, текстуры, степлеры) и оставила всё, что в нём было. Поверх добавлено:

- сборка через **CMake + vcpkg**, так что проект собирается на любом компьютере, а не только у автора;
- исправлены найденные ошибки;
- **job system на enkiTS** (п. 2.1);
- задача №2: **параллельная подготовка отрисовки и анимации** на стресс-сцене из 20 000 кубов (п. 2.3);
- **Трасу и замеры до/после** (п. 2.4).

Базовые примитивы (треугольник, квадрат, куб) остались и работают.

Внимание. П. 2.2 (асинхронная загрузка ресурсов, L1) не сделан. Менеджер ресурсов в 3lab есть, но грузит всё синхронно. Всё нужное для 2.2 в job system уже есть, см. раздел 6.

Куда класть в репозиторий: это содержимое папки Engine--3/Engine-main/ ветки 3lab. Папку glm/ в репозитории нужно удалить, glm теперь ставит vcpkg. Инструкция в корневом README.md («скопируйте путь к glm в свойства проекта») больше не нужна, актуальная лежит в README.md рядом с этим файлом.

Кнопки: что можно нажать

Проект открывается в Visual Studio через «Файл → Открыть → Папка». Цель **Engine.exe**, запуск **F5**. Первая конфигурация идёт 5–10 минут: vcpkg собирает библиотеки.

Везде

Кнопка	Что делает
W	показать или спрятать второе окно с крутящимся квадратом
F5 / F6	сохранить / загрузить сцену (scene.json)
Esc	выйти из программы

Главное меню (виден только фон, сцена скрыта)

Кнопка	Что делает
Enter	начать игру

Игра

Кнопка	Что делает
↑ ↓ ← →	двигать камеру вперёд, назад, влево, вправо
Q / E	камера вниз / вверх
ПКМ + мышь	вращать камеру вокруг цели
ЛКМ + мышь вверх/вниз	приблизить / отдалить
Колесо мыши	приблизить / отдалить (раньше не работало)
B	включить / выключить стресс-сцену: 20 000 анимированных кубов с текстурами
J	job system: параллельно / на одном потоке (FPS в заголовке окна)
P	пауза

Пауза

Кнопка	Что делает
P	продолжить
Backspace	выйти в главное меню

Второе окно

Его можно растягивать мышью. Крестик прячет окно, а не закрывает игру; вернуть его — снова **W**.

Совет для демо: Enter, затем **B**, затем несколько раз **J**, следя за FPS и счётчиком видимых объектов в заголовке окна.

Запуск из командной строки

Аргумент	Что делает
<code>--bench</code>	автоматический замер: сразу игра со стресс-сценой, прогрев, N кадров, результат в CSV, выход
<code>--jobs=on</code> / <code>--jobs=off</code>	с job system / без неё (замер «до»)
<code>--workers=N</code>	число рабочих потоков (по умолчанию ядра – 1)
<code>--entities=N</code>	число кубов в стресс-сцене
<code>--warmup=N</code> , <code>--frames=N</code>	кадры прогрева и замера
<code>--out=файл.csv</code> , <code>--label=текст</code>	куда писать результат и как подписать прогон
<code>--vsync=on</code>	включить вертикальную синхронизацию

Готовый замер «до/после» (3 + 3 прогона, сводная таблица): `bench.ps1` рядом с `Engine.exe` в `out\build\x64-release`.

1. Сборка: vcpkg + CMake вместо .sln/.vcxproj

	Было	Стало
Сборка	<code>Init Direct3D.sln</code> / <code>.vcxproj</code>	<code>CMakeLists.txt</code> + <code>CMakePresets.json</code> (VS: «Открыть папку»)
Assimp	поставлен вручную в <code>C:\Program Files\Assimp</code> , путь к <code>.lib</code> прописан в проекте	<code>vcpkg</code>
glm	папка <code>glm/</code> в репозитории (около 1700 файлов), путь <code>C:\Users\Анна\Documents\...</code> в свойствах проекта; каждому надо было править его под себя	<code>vcpkg</code>
<code>stb_image, json</code>	скопированные файлы (<code>stb_image.h</code> в двух местах, <code>json.hpp</code>)	<code>vcpkg</code> (<code>stb</code> , <code>nlohmann-json</code>)
Конфигурации	пути настроены только для Debug x64, Release не собирался	Debug и Release (x64)
Версии библиотек	—	зафиксированы полем <code>builtin-baseline</code> в <code>vcpkg.json</code> : у всех одинаковые
Файлы рядом с exe	—	<code>config.json</code> , <code>scene.json</code> , <code>Shaders/</code> , <code>assets/</code> копируются при сборке; движок при старте переходит в папку exe

Удалено: `.sln`, `.vcxproj*`, `.suo`, `x64/` и `Init Direct3D/` (сборочный мусор), `engine.log`, `json.hpp`, оба `stb_image.h`, `external/`, `tiny_obj_loader.h` и `MeshLoader.h` (он был целиком закомментирован: модели грузит Assimp).

Кодировки: 9 файлов, включая оба шейдера, были в cp1251, поэтому в VS и в других редакторах вместо комментариев шли кракозябры. Теперь все файлы в UTF-8.

2. Исправленные ошибки

Проблема	Что было	Исправление
Upload-буферы мешей	один буфер-член адаптера на все меши: загрузка второго меша освобождала буфер первого до того, как GPU его скопировал	буферы хранятся в GPUmesh; меши грузятся на GPU при загрузке сцены (UploadMeshToGPU), а не посреди кадра в Draw
Примитивы без текстуры	пиксельный шейдер всегда читает t0, но для примитивов (квадрат во втором окне) таблица дескрипторов не привязывалась	белая текстура 1×1 в слоте 0; шейдер возвращает текстура × цвет вершины
Примитив «куб»	рисовал 36 индексов из буфера, где их 9 (чтение за границей)	добавлена геометрия куба
DrawPrimitiveECS	из матрицы модели брались только сдвиг, поворот вокруг Z и масштаб по X	используется полная матрица
Лог-спам	DrawMesh без текстуры писал WARNING на каждый объект каждый кадр	белая текстура, без предупреждений
Колесо мыши	m_scrollDelta нигде не заполнялся, зум колесом не работал	обработка WM_MOUSEWHEEL
Ввод	GetAsyncKeyState ловил клавиши и мышь, даже когда окно не в фокусе; Sleep(200) «от залипания» замораживал весь цикл	класс Input из оконных сообщений; WasKeyPressed срабатывает один раз на нажатие
Второе окно закрывало игру	та же оконная процедура, что у главного: крестик → WM_DESTROY → выход	своя SecondaryWndProc: крестик прячет окно
Ресайз второго окна	его WM_SIZE менял размер swap chain главного окна	второе окно ресайзит свой swap chain
Второе окно: viewport и глубина	viewport и буфер глубины главного окна; swap chain 400×400 при клиентской области меньше	свой viewport и свой буфер глубины; размер по клиентской области
Второе окно: камера	квадрат рисовался с видом и проекцией главной камеры	своя (единичная) камера
Фокус	клик по второму окну ставил игру на паузу	переход во второе окно паузу не включает
Смена состояний	ChangeState / ClearStates удаляли состояние посреди его же Update() (use-after-free)	переходы откладываются до конца Update
mCurrBackBuffer++	счётчик увеличивался вручную и мог разойтись со swap chain	берётся GetCurrentBackBufferIndex()
F2	в D3DApp включает 4xMSAA, который swap chain FLIP_DISCARD не поддерживает: падение	F2 перехватывается
Кадр = частота монитора	без tearing flip-модель ограничивает FPS частотой монитора даже при Present(0), и замеры упираются в 1/Гц	tearing при выключенном vsync
Шейдеры	всегда компилировались с DEBUG и без оптимизаций (и в Release); путь «расширялся» побайтно	флаги отладки только в Debug; путь переводится из UTF-8
Нет шейдеров	ошибка только логировалась, дальше падение на nullptr в BuildPSO	инициализация прерывается с понятной ошибкой
UploadTextureToGPU	при ошибке command list оставался открытым, и следующий Reset падал	закрывается всегда; переполнение хипа SRV (64) проверяется
ResourceManager::Clear	не очищал кэш текстур	очищает
Удаление сущностей	DestroyEntity проходит весь список на каждую сущность: удаление тысяч занимало секунды	World::DestroyEntities (пакетно)
Отбраковка	каждый кадр пересоздавалась SpatialGrid со строковыми ключами, отсекались только объекты дальше 30 от камеры (объекты за спиной рисовались)	отбраковка по пирамиде видимости, параллельно (см. п. 3). SpatialGrid остался в World, но в кадре не используется
Логгер	открывал файл на каждую строку, не потокобезопасный, wstring превращал кириллицу в мусор	файл открыт всё время, мьютекс (логгер зовётся из задач), UTF-8, время и id потока

Проблема	Что было	Исправление
Конфиг	JSON разбирался вручную, <code>std::stof</code> падал на опечатке	<code>nlohmann-json</code> ; при ошибке — лог и значения по умолчанию; аргументы командной строки
Common/	конструкции, которые понимает только MSVC (<code>&CD3DX12_...()</code> , <code>L#x</code> , <code>ifstream(wstring)</code>)	переписаны стандартно, на MSVC работает так же

3. Новое

Job system — `Jobs/JobSystem.h/.cpp` (п. 2.1)

Подсистема движка поверх **enkiTS**: пул потоков, lock-free очереди, work stealing, приоритеты.

- `Initialize / Shutdown` в `Application`. Job system инициализируется **первой** и останавливается **первой**: живые задачи дорабатывают, новые не принимаются.
- Единая точка постановки задач:
 - `ParallelFor(count, minRange, fn)` — диапазоны на пуле, вызывающий поток тоже работает;
 - `Submit(fn, JobPriority, name) → JobHandle, Wait(handle)` — фоновые задачи;
 - `RunOnMainThread(fn) + PumpMainThread(лимит)` — очередь главного потока с лимитом на кадр (8).
- **Приоритеты**: ожидая `ParallelFor`, главный поток берёт только задачи High. Долгая Low-задача (загрузка) не выполнится внутри кадра.
- `--jobs=off` или клавиша **J** — всё выполняется на одном потоке. Это режим «до» в том же exe.
- Tracy: имена потоков `Job Worker N`, зоны ожидания `Wait for task / Idle`, графики `Jobs in flight`, `Main-thread queue`.

Задача №2 (п. 2.3): что именно сделано

Какая задача выбрана. Кандидат из списка ТЗ «распараллеливание внутри одной системы». ТЗ описывает его так: «снимок сущностей в массив → dispatch по диапазонам, работает и с хеш-таблицами, нужна сцена с тысячами сущностей, сделайте замерочную». Наш `World` хранит компоненты в `unordered_map`, так что это ровно тот случай.

Что перенесено на job system.

1. Подготовка отрисовки — `RenderSystem::Render` (`RenderSystem.cpp`):

- снимок списка всех сущностей режется на куски по 256;
- каждый кусок выполняется задачей job system: читает `Tag`, `Transform` и `MeshRenderer` из хеш-таблиц, считает мировую матрицу, проверяет, попадает ли объект в камеру (отбраковка по пирамиде видимости), и пишет готовый `DrawItem`;
- главный поток склеивает куски по порядку (порядок отрисовки не меняется) и одним вызовом `DrawItems` записывает команды для GPU. Вид и проекция задаются один раз, а не на каждый объект.

Раньше всё это делалось последовательно прямо в `Application::Draw`, по одному объекту за раз.

2. Анимация стресс-сцены — `UpdateStressAnimation`: кубы вращаются и покачиваются, и эта работа тоже раскладывается по потокам. Каждый куб пишет только свой `Transform`, поэтому блокировки не нужны.

Пока идёт подготовка, мир только читается: добавлять и удалять сущности в это время нельзя, поэтому потоки не мешают друг другу.

«Замерочная» сцена. Клавиша **B** или запуск с `--bench: 20 000` кубов (`cube.obj`) с тремя текстурами, сетка под степлерами. Seed фиксированный, поэтому в каждом замере сцена одна и та же.

Как показать «до/после».

- Клавиша **J** на лету переключает job system: параллельно или на одном потоке. В заголовке окна видны FPS и число видимых объектов.
- Для замера: `Engine.exe --bench --jobs=off` или `--jobs=on`, либо сразу `bench.ps1`.

Результат (Release, 20 000 кубов, 3 + 3 прогона по 2000 кадров, подробно в разделе 5):

	без job system	с ней	ускорение
подготовка отрисовки	1.52 мс	0.27 мс	×5.6
анимация	0.59 мс	0.14 мс	×4.2
кадр, медиана	3.92 мс	1.64 мс	×2.4
кадр, p99	8.42 мс	4.24 мс	

Зоны отмечены в Tracy (RenderSystem::Prepare, StressAnimation, ParallelFor range), поэтому на трейсе видно, как работа расходится по потокам Job Worker N.

Что осталось команде.

- **Согласовать выбор задачи в чате ММ** (так требует ТЗ). Например: «параллельная подготовка отрисовки и анимация в ECS на сцене из 20 000 сущностей».
- Для отчёта перемерить на свободном компьютере и **снять трейсы Tracy** «до» (J выключен) и «после» (J включён).

Замеры (п. 2.4) — [Benchmark.h/.cpp](#), [tools/bench.ps1](#)

--bench: фиксированная сцена и камера, прогрев, N кадров. Считаются медиана, p95 и p99 кадра и отдельно зон StressAnimation, RenderPrepare, RenderRecord; результат дописывается в CSV. bench.ps1 запускает 3 прогона без job system и 3 с ней и строит сводную таблицу.

Диагностика

- CrashHandler.h: при падении в engine.log пишется код исключения и стек вызовов с файлами и строками (для Release тоже собирается .pdb).
- Debug: ошибки и предупреждения D3D12 debug layer попадают в engine.log.
- В заголовке окна: состояние, режим job system, число видимых и всех объектов, FPS.

4. Карта файлов (Chapter 4 Direct3D Initialization/Init Direct3D/)

Файл	Статус
Jobs/JobSystem.*, RenderSystem.cpp, Benchmark.*, ConfigManager.cpp, Input.h, Profiler.h, CrashHandler.h	новые
InitDirect3DApp.cpp	WinMain и Application; методы D3D12RenderAdapter из него перенесены в D3D12RenderAdapter.cpp
D3D12RenderAdapter.*, RenderAdapter.h, RenderSystem.h, Application.h, GameStateManager.h, Logger.h, ConfigManager.h, config.json	переписаны или сильно изменены (старый интерфейс сохранён)
World.h, MeshData.*, GPUMesh.h, ResourceManager.h, PlayState.h, PauseState.h, SceneSerializer.h, Shaders/PixelShader.hlsl	точечные правки
MeshLoader.h, tiny_obj_loader.h, json.hpp, stb_image.h, external/	удалены
корень: CMakeLists.txt, CMakePresets.json, vcpkg.json, tools/bench.ps1, README.md	новые

5. Как проверялось и результаты

- **Visual Studio 2026 (MSVC 19.51) + vcpkg:** Debug и Release собираются с **0 предупреждений**.
- **Сценарий на Debug под D3D12 debug layer и на Release:** меню → игра → камера (стрелки, Q/E, ПКМ, ЛКМ, колесо) → стресс-сцена → J → второе окно (ресайз, крестик) → ресайз главного окна → пауза → В выкл/вкл → F5/F6 → меню → снова игра → F2 → Esc. **0 ошибок и предупреждений** (в том числе от debug layer), код выхода 0.
- **Длинная сессия:** 30 000 кадров со стресс-сценой, выход 0, память стабильна (около 75 МБ).
- **Тест job system** (20 итераций): покрытие ParallelFor, фоновые Low-задачи параллельно с кадрами (главный поток ни разу не взял Low-задачу), wait, вложенные задачи, исключение в задаче, **шатдаун с 32 живыми**

задачами (около 16 мс, без зависаний).

- **Замер** (Release, 20 000 кубов, 3 + 3 прогона по 2000 кадров, 12 потоков):

метрика	без job system	с job system	ускорение
кадр, медиана	3.92 мс	1.64 мс	×2.4
кадр, p95	5.75 мс	2.66 мс	
кадр, p99	8.42 мс	4.24 мс	
StressAnimation	0.59 мс	0.14 мс	×4.2
RenderPrepare	1.52 мс	0.27 мс	×5.6
RenderRecord (главный поток)	0.23 мс	0.18 мс	—

Для отчёта **перемерьте на свободном компьютере** (bench.ps1) и снимите трейсы Trasy «до» и «после».

6. Для п. 2.2 (асинхронная загрузка) — что уже готово

```
// чтение файла + разбор (Assimp / stb_image) - в фоне, низкий приоритет
jobs.Submit([=] {
    if (jobs.IsShuttingDown()) return; // корректный шатдаун
    auto mesh = MeshData::LoadFromFile(path); // CPU-работа в воркере
    jobs.RunOnMainThread([=] { d3d->UploadMeshToGPU(*mesh); }); // финализация на главном потоке
}, JobPriority::Low, "LoadMesh");
// Application::Update вызывает PumpMainThread(8) каждый кадр - это "памп с лимитом на кадр"
```

Placeholder тоже есть: если у меша ещё нет GPU-версии, `RenderSystem` рисует вместо него примитив «куб». Осталось:

- сделать `ResourceManager` потокобезопасным (мьютекс на кэш);
- перевести `InitScene` на фоновую загрузку;
- снять трейс «фриз до → ровный кадр после» (`--jobs=off` превращает `Submit` в синхронный вызов, это и есть «до»).